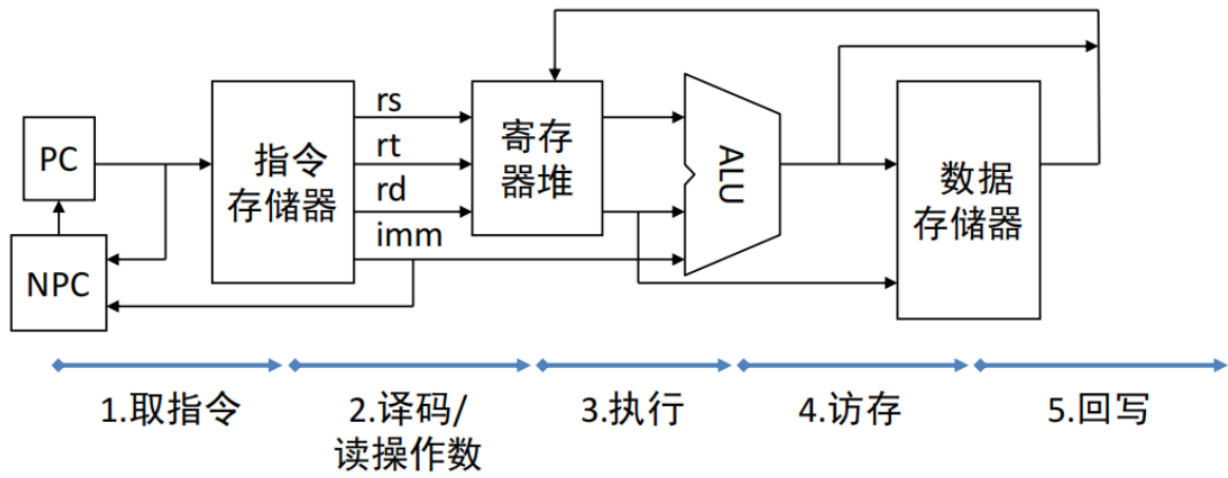


# CO-P3设计文档

P3课下测试需要考虑的：add, sub, ori, lw, sw, beq, lui, nop 我额外实现的指令：and,lb,jjr,jal,jalr等等



对给出的数据样例的解读：

| 32 位二进制 code                           | MIPS 指令              |
|----------------------------------------|----------------------|
| 001101 00000 11100 000000000000000000  | ori \$28, \$1, 0     |
| 001101 00000 11101 000000000000000000  | ori \$29, \$1, 0     |
| 001101 00000 00001 0011010001010110    | ori \$1, \$0, 0x3456 |
| 000000 00001 00001 0000100000100000    | add \$1, \$1, \$1    |
| 100011 00000 00001 000000000000000100  | lw \$1, 0x4(\$0)     |
| 101011 00000 00001 000000000000000100  | sw \$1, 0x4(\$0)     |
| 001111 00000 00010 0111100001111000    | lui \$2, 0x7878      |
| 000000 00010 00001 0001100000100010    | sub \$3, \$2, \$1    |
| 001111 00000 00101 0001001000110100    | lui \$5, 0x1234      |
| 001101 00000 00100 000000000000000101  | ori \$4, \$0, 5      |
| 000000 00000 00000 000000000000000000  | nop                  |
| 101011 00100 00101 111111111111111111  | sw \$5, -1(\$4)      |
| 100011 00100 00011 111111111111111111  | lw \$3, -1(\$4)      |
| 000100 00011 00101 000000000000000001  | beq \$3, \$5, 4      |
| 000100 00000 00000 0000000000000001101 | beq \$0, \$0, 0x34   |
| 001101 00011 00111 000001000000000100  | ori \$7, \$3, 0x0404 |
| 000100 00111 00011 0000000000000001011 | beq \$7, \$3, 0x2c   |
| 000000 00000 00000 000000000000000000  | nop                  |

| 32 位二进制 code                        | MIPS 指令              |
|-------------------------------------|----------------------|
| 001111 00000 01000 0111011101110111 | lui \$8, 0x7777      |
| 001101 01000 01000 1111111111111111 | ori \$8, \$8, 0xffff |
| 000000 00000 01000 0000000000100010 | sub \$0, \$0, \$8    |
| 001101 00000 00000 0001000100000000 | ori \$0, \$0, 0x1100 |
| 000000 00111 00110 0101000000100000 | add \$10, \$7, \$6   |
| 001101 00000 01000 0000000000000000 | ori \$8, \$0, 0      |
| 001101 00000 01001 0000000000000001 | ori \$9, \$0, 1      |
| 000000 01000 01010 0100000000100000 | add \$8, \$8, \$10   |
| 000100 01000 01001 1111111111111110 | beq \$8, \$9, -8     |
| 000100 00000 00000 1111111111111111 | beq \$0, \$0, -4     |

## Controller的输出端口的含义

**NPCslt[1:0]** 0:PC+4 1:PC+4+imm16 2:PC+imm26 3:\$ra **grfWE** 决定是否回写入grf **EXTOp** 0: 无符号 1: 符号  
扩展 **ALUSrc** 决定ALU中的inputB输入: 0:rt寄存器中的值 1:imm16 **RegDst** 回写到哪个寄存器? 0: rt 1: rd  
**MemtoReg** 0: 非lw的回写 1: lw回写 **DMWE** 决定DM的写使能 **type** 1add, 2sub, 3ori, 4lw, 5sw, 6beq, 7lui

| 控制信号     | add     | sub     | ori     | lw      | sw      | beq     | lui     |
|----------|---------|---------|---------|---------|---------|---------|---------|
| op       | 00 0000 | 00 0000 | 00 1101 | 10 0011 | 10 1011 | 00 0100 | 00 1111 |
| func     | 10 0000 | 10 0010 | X       | X       | X       | X       | X       |
| NPC_sel  | 0       | 0       | 0       | 0       | 0       | 1       | 0       |
| jrfWE    | 1       | 1       | 1       | 1       | 0       | 0       | 1       |
| ExtOp    | X       | X       | 0       | 1       | 1       | X       | X       |
| ALUSrc   | 0       | 0       | 1       | 1       | 1       | 0       | 1       |
| RegDst   | 1       | 1       | 0       | 0       | X       | X       | 0       |
| MemtoReg | 0       | 0       | 0       | 1       | X       | X       | 0       |
| DMWE     | 0       | 0       | 0       | 0       | 1       | 0       | 0       |

## 实验模块

### NPC

| 信号名         | 描述           |
|-------------|--------------|
| PC[31:0]    | 当前指令的地址      |
| imm16[15:0] | 16位偏移 offset |

| 信号名         | 描述                                    |
|-------------|---------------------------------------|
| imm26[25:0] | 26位偏移 jal和j指令                         |
| \$ra        | 32位的\$ra jr                           |
| zero        | rs与rt相等的比较结果                          |
| NPCslt[1:0] | 0:PC+4 1:PC+4+imm16 2:PC+imm26 3:\$ra |
| NPC[31:0]   | 下一条指令的地址                              |

grf

| 信号名      | 描述          |
|----------|-------------|
| A1[31:0] | 编号rs        |
| A2[31:0] | 编号rt        |
| A3[31:0] | 编号rd        |
| WD[31:0] | lw sw回写的值   |
| WE       | lw sw回写使能信号 |
| clk Re   |             |

ALU

| 信号名          | 描述            |
|--------------|---------------|
| inputA[31:0] | \$rs          |
| inputB[31:0] | \$rt or imm16 |
| type[3:0]    | 种类            |
| output[31:0] | ALU的输出        |

IM something

PC something

DM

| 信号名        | 描述        |
|------------|-----------|
| Addr[31:0] | 存入内存的地址   |
| DMWD[31:0] | 存入内存的值    |
| DMWE       | 仅sw时有效    |
| clk Re     |           |
| out[31:0]  | 回写的值 (lw) |

| 信号名        | 描述        |
|------------|-----------|
| byte[31:0] | 回写的值 (lb) |

## 两个代码

### mips代码生成注释的C++程序

```
#include <vector>
#include <string>
#include <sstream>
#include <unordered_map>
#include <algorithm>
#include <cstdint>

using namespace std;

// 寄存器别名映射表
unordered_map<string, string> regAlias = {
    {"$zero", "$0"}, {"$at", "$1"}, {"$v0", "$2"}, {"$v1", "$3"}, {"$a0", "$4"},
    {"$a1", "$5"}, {"$a2", "$6"}, {"$a3", "$7"}, {"$t0", "$8"}, {"$t1", "$9"}, {"$t2",
"$10"}, {"$t3", "$11"}, {"$t4", "$12"}, {"$t5", "$13"}, {"$t6", "$14"}, {"$t7",
"$15"}, {"$s0", "$16"}, {"$s1", "$17"}, {"$s2", "$18"}, {"$s3", "$19"}, {"$s4",
"$20"}, {"$s5", "$21"}, {"$s6", "$22"}, {"$s7", "$23"}, {"$t8", "$24"}, {"$t9",
"$25"}, {"$k0", "$26"}, {"$k1", "$27"}, {"$gp", "$28"}, {"$sp", "$29"}, {"$fp",
"$30"}, {"$ra", "$31"};

// 清理指令行 (去除注释、多余空格)
string cleanLine(const string &line)
{
    size_t commentPos = line.find('#');
    string clean = (commentPos != string::npos) ? line.substr(0, commentPos) :
line;
    clean.erase(0, clean.find_first_not_of(" \t"));
    clean.erase(clean.find_last_not_of(" \t") + 1);
    replace(clean.begin(), clean.end(), '\t', ' ');
    string res;
    bool inSpace = false;
    for (char c : clean)
    {
        if (c == ' ')
        {
            if (!inSpace)
            {
                res += ' ';
                inSpace = true;
            }
        }
        else
        {
            res += c;
            inSpace = false;
        }
    }
}
```

```

    }
}
return res;
}

// 解析寄存器名 (处理别名)
string parseReg(const string &reg)
{
    if (regAlias.count(reg))
        return regAlias[reg];
    if (reg.size() >= 2 && reg[0] == '$' && isdigit(reg[1]))
        return reg;
    return "";
}

// 解析立即数 (支持十进制、十六进制0x前缀)
int32_t parseImm(const string &immStr)
{
    if (immStr.empty())
        return 0;
    int base = 10;
    size_t start = 0;
    if (immStr.substr(0, 2) == "0x")
    {
        base = 16;
        start = 2;
    }
    return stoi(immStr.substr(start), nullptr, base);
}

// 从内存地址字符串中提取偏移量 (如"4($0)"提取4)
int32_t parseMemOffset(const string &memStr)
{
    size_t pos = memStr.find('(');
    if (pos == string::npos)
        return 0;
    string offsetStr = memStr.substr(0, pos);
    return parseImm(offsetStr);
}

// 从内存地址字符串中提取基址寄存器 (如"4($0)"提取$0)
string parseMemBaseReg(const string &memStr)
{
    size_t pos1 = memStr.find('(');
    size_t pos2 = memStr.find(')');
    if (pos1 == string::npos || pos2 == string::npos)
        return "";
    string regStr = memStr.substr(pos1 + 1, pos2 - pos1 - 1);
    return parseReg(regStr);
}

// 解析一行指令, 返回被修改的寄存器及新值 (键: 寄存器名, 值: 新值)
unordered_map<string, int32_t> getModifiedRegistersWithValue(
    const string &line,

```

```

unordered_map<string, int32_t> &regs, // 当前寄存器状态
unordered_map<int32_t, int32_t> &mem, // 内存状态
int32_t &pc                          // 程序计数器
)
{
    unordered_map<string, int32_t> modified;
    string clean = cleanLine(line);
    if (clean.empty())
        return modified;

    istringstream iss(clean);
    vector<string> tokens;
    string token;
    while (iss >> token)
    {
        size_t pos = 0;
        while ((pos = token.find(',')) != string::npos)
        {
            if (pos > 0)
                tokens.push_back(token.substr(0, pos));
            token = token.substr(pos + 1);
            token.erase(0, token.find_first_not_of(" \t"));
        }
        if (!token.empty())
            tokens.push_back(token);
    }
    if (tokens.empty())
        return modified;

    string op = tokens[0];
    int32_t nextPC = pc + 4; // 下一条指令地址 (默认)

    // 1. 算术逻辑指令 (add, sub, and, or等) : op $d, $s, $t
    if (op == "add" || op == "addu" || op == "sub" || op == "subu" ||
        op == "and" || op == "or" || op == "xor" || op == "nor" ||
        op == "slt" || op == "sltu")
    {
        if (tokens.size() >= 4)
        {
            string d = parseReg(tokens[1]);
            string s = parseReg(tokens[2]);
            string t = parseReg(tokens[3]);
            if (d.empty() || s.empty() || t.empty() || d == "$0")
                return modified;

            int32_t valS = regs[s];
            int32_t valT = regs[t];
            int32_t res = 0;

            if (op == "add" || op == "addu")
                res = valS + valT;
            else if (op == "sub" || op == "subu")
                res = valS - valT;
            else if (op == "and")

```

```

        res = valS & valT;
    else if (op == "or")
        res = valS | valT;
    else if (op == "xor")
        res = valS ^ valT;
    else if (op == "nor")
        res = ~(valS | valT);
    else if (op == "slt")
        res = (valS < valT) ? 1 : 0;
    else if (op == "sltu")
        res = ((uint32_t)valS < (uint32_t)valT) ? 1 : 0;

    regs[d] = res;
    modified[d] = res;
}
}

// 2. 立即数指令 (addi, andi, ori等) : op $t, $s, imm
else if (op == "addi" || op == "addiu" || op == "andi" || op == "ori" ||
        op == "xori" || op == "slti" || op == "sltiu")
{
    if (tokens.size() >= 3)
    {
        string t = parseReg(tokens[1]);
        string s = parseReg(tokens[2]);
        if (t.empty() || s.empty() || t == "$0")
            return modified;

        int32_t valS = regs[s];
        int32_t imm = parseImm(tokens[3]);
        int32_t res = 0;

        if (op == "addi" || op == "addiu")
            res = valS + imm;
        else if (op == "andi")
            res = valS & (imm & 0xFFFF); // 零扩展
        else if (op == "ori")
            res = valS | (imm & 0xFFFF); // 零扩展
        else if (op == "xori")
            res = valS ^ (imm & 0xFFFF); // 零扩展
        else if (op == "slti")
            res = (valS < imm) ? 1 : 0;
        else if (op == "sltiu")
            res = ((uint32_t)valS < (uint32_t)imm) ? 1 : 0;

        regs[t] = res;
        modified[t] = res;
    }
}

// 3. 加载指令 (lw, lb, lui等) : op $t, imm($s) 或 lui $t, imm
else if (op == "lw" || op == "lb")
{ // 简化: 假设lb与lw效果相同 (实际有符号扩展, 这里简化)
    if (tokens.size() >= 3)

```

```

    {
        string t = parseReg(tokens[1]);
        if (t.empty() || t == "$0")
            return modified;

        int32_t offset = parseMemOffset(tokens[2]);
        string s = parseMemBaseReg(tokens[2]);
        if (s.empty())
            return modified;

        int32_t addr = regs[s] + offset;
        int32_t val = mem.count(addr) ? mem[addr] : 0; // 内存地址未初始化则为0

        regs[t] = val;
        modified[t] = val;
    }
}
else if (op == "lui")
{ // lui $t, imm: 高16位加载, 低16位为0
    if (tokens.size() >= 3)
    {
        string t = parseReg(tokens[1]);
        if (t.empty() || t == "$0")
            return modified;

        int32_t imm = parseImm(tokens[2]);
        int32_t res = (imm & 0xFFFF) << 16; // 高16位

        regs[t] = res;
        modified[t] = res;
    }
}

// 4. 存储指令 (sw, sb) : 不修改寄存器, 仅更新内存
else if (op == "sw" || op == "sb")
{
    if (tokens.size() >= 3)
    {
        string t = parseReg(tokens[1]);
        if (t.empty())
            return modified;

        int32_t offset = parseMemOffset(tokens[2]);
        string s = parseMemBaseReg(tokens[2]);
        if (s.empty())
            return modified;

        int32_t addr = regs[s] + offset;
        mem[addr] = regs[t]; // 写入内存
    }
}

// 5. 乘除指令 (mult, div, mfhi, mflo等)
else if (op == "mult" || op == "multu" || op == "div" || op == "divu")

```



```

{
    if (tokens.size() >= 3)
    {
        string s = parseReg(tokens[1]);
        string t = parseReg(tokens[2]);
        if (s.empty() || t.empty())
            return modified;

        int32_t valS = regs[s];
        int32_t valT = regs[t];
        int64_t temp = 0;

        if (op == "mult")
            temp = (int64_t)valS * (int64_t)valT;
        else if (op == "multu")
            temp = (uint64_t)(uint32_t)valS * (uint64_t)(uint32_t)valT;
        else if (op == "div")
        {
            if (valT != 0)
                temp = (int64_t)(valS / valT) | ((int64_t)(valS % valT) <<
32);
            }
        else if (op == "divu")
        {
            if (valT != 0)
                temp = (int64_t)((uint32_t)valS / (uint32_t)valT) |
                    ((int64_t)((uint32_t)valS % (uint32_t)valT) << 32);
            }

        regs["HI"] = (int32_t)(temp >> 32);          // 高32位
        regs["LO"] = (int32_t)(temp & 0xFFFFFFFF); // 低32位
        modified["HI"] = regs["HI"];
        modified["LO"] = regs["LO"];
    }
}
else if (op == "mfhi")
{ // mfhi $d: HI -> $d
    if (tokens.size() >= 2)
    {
        string d = parseReg(tokens[1]);
        if (d.empty() || d == "$0")
            return modified;

        regs[d] = regs["HI"];
        modified[d] = regs["HI"];
    }
}
else if (op == "mflo")
{ // mflo $d: LO -> $d
    if (tokens.size() >= 2)
    {
        string d = parseReg(tokens[1]);
        if (d.empty() || d == "$0")
            return modified;
    }
}

```

```

        regs[d] = regs["LO"];
        modified[d] = regs["LO"];
    }
}
else if (op == "mthi")
{ // mthi $s: $s -> HI
    if (tokens.size() >= 2)
    {
        string s = parseReg(tokens[1]);
        if (s.empty())
            return modified;

        regs["HI"] = regs[s];
        modified["HI"] = regs[s];
    }
}
else if (op == "mtlo")
{ // mtlo $s: $s -> LO
    if (tokens.size() >= 2)
    {
        string s = parseReg(tokens[1]);
        if (s.empty())
            return modified;

        regs["LO"] = regs[s];
        modified["LO"] = regs[s];
    }
}

// 6. 跳转指令 (jal修改$ra)
else if (op == "jal")
{
    regs["$31"] = nextPC; // $ra = 下一条指令地址
    modified["$31"] = nextPC;
    // 跳转不影响寄存器值计算, 仅更新PC (此处简化)
}

// 7. 伪指令 (move, li)
else if (op == "move")
{ // move $d, $s: $d = $s
    if (tokens.size() >= 3)
    {
        string d = parseReg(tokens[1]);
        string s = parseReg(tokens[2]);
        if (d.empty() || s.empty() || d == "$0")
            return modified;

        regs[d] = regs[s];
        modified[d] = regs[s];
    }
}
else if (op == "li")
{ // li $t, imm: $t = imm

```

```

        if (tokens.size() >= 3)
        {
            string t = parseReg(tokens[1]);
            if (t.empty() || t == "$0")
                return modified;

            int32_t imm = parseImm(tokens[2]);
            regs[t] = imm;
            modified[t] = imm;
        }
    }

    // 8. 空指令 (nop) : 无操作
    else if (op == "nop")
    {
    }

    // 更新PC (分支/跳转指令的PC更新简化处理, 不影响寄存器值)
    pc = nextPC;

    return modified;
}

int main()
{
    // 初始化寄存器状态 ($0固定为0, 其他初始为0, HI/LO初始为0)
    unordered_map<string, int32_t> regs;
    for (int i = 0; i <= 31; ++i)
        regs["$" + to_string(i)] = (i == 0) ? 0 : 0;
    regs["HI"] = 0;
    regs["LO"] = 0;

    // 初始化内存 (数据存储器)
    unordered_map<int32_t, int32_t> mem;

    // 程序计数器 (初始指令地址为0)
    int32_t pc = 0;

    // 读取输入
    cout << "请输入MIPS汇编代码 (空行结束) : " << endl;
    string line;
    vector<string> code;
    while (getline(cin, line))
    {
        if (line.empty())
            break;
        code.push_back(line);
    }

    // 分析并输出结果
    cout << "\n分析结果 (每行指令修改的寄存器及值) : " << endl;
    for (size_t i = 0; i < code.size(); ++i)
    {
        auto modified = getModifiedRegistersWithValue(code[i], regs, mem, pc);
    }
}

```

```

        cout << "行" << i + 1 << ": " << code[i] << " → ";
        if (modified.empty())
        {
            cout << "无寄存器被修改";
        }
        else
        {
            size_t cnt = 0;
            for (auto &[reg, val] : modified)
            {
                if (cnt > 0)
                    cout << ", ";
                cout << reg << "=" << val;
                cnt++;
            }
        }
        cout << endl;
    }

    return 0;
}

```

## 机器码转mips程序

```

#include <bits/stdc++.h>

using namespace std;

bool overflowtest = true;
//if true, add/sub/addi; else if false, addu/subu/addiu

// hex string to bin string(32 bits)
string hexToBin(const string& hex) {
    string binary;
    unsigned int temp;
    sscanf(hex.c_str(), "%x", &temp);
    while (temp) {
        char c = temp % 2 + '0';
        binary = string(1, c) + binary;
        temp /= 2;
    }
    int len = binary.size();
    binary = string(32 - len, '0') + binary;
    return binary;
}

// bin string to dec string(unsigned)
string ubinToDec(const string& bin) {
    string decimal;
    bitset<32> bitset(bin);
    unsigned int temp = static_cast<unsigned int>(bitset.to_ulong());
}

```

```

        decimal = to_string(temp);
        return decimal;
    }

// bin string to dec string(signed)
string binToDec(const string& bin) {
    int len = bin.size();
    string decimal;
    bitset<32> bitset(bin);
    int temp = static_cast<int>(bitset.to_ulong());
    if (bin[0] == '1') {
        temp = temp - (2 << (len - 1));
    }
    decimal = to_string(temp);
    return decimal;
}

// bin string to dec int(signed)
int binToDecInt(const string& bin) {
    int len = bin.size();
    string decimal;
    bitset<32> bitset(bin);
    int temp = static_cast<int>(bitset.to_ulong());
    if (bin[0] == '1') {
        temp = temp - (2 << (len - 1));
    }
    return temp;
}

int main() {
    ifstream inputFile("input.txt");
    ofstream outputFile("output.txt");
    string line;

    // get hex codes in line
    vector<string> binaryArray;
    vector<string> mipsCodes;
    vector<int> labels;
    string label = "label";
    int labelcnt = 0;
    while (getline(inputFile, line)) {
        // erase the blank characters at beginning or endding of a line
        line.erase(0, line.find_first_not_of(" \t\n\r\f\v"));
        line.erase(line.find_last_not_of(" \t\n\r\f\v") + 1);

        // switch line from hex to bin, then push it in binaryArray
        if (!line.empty()) {
            // cout << line << "\n";
            string binary = hexToBin(line);
            binaryArray.push_back(binary);
        }
    }

    inputFile.close();

```

```

// print bin codes(using it to debug)
for (string whichline : binaryArray) {
    cout << whichline << "\n";
}

// core: translate bin codes into MIPS codes
for (string whichline : binaryArray) {
    string mipsline;
    string OpCode = whichline.substr(0, 6);
    string rs = whichline.substr(6, 5);
    string rt = whichline.substr(11, 5);
    string rd = whichline.substr(16, 5);
    string Funct = whichline.substr(26, 6);
    string imm16 = whichline.substr(16, 16);
    string imm26 = whichline.substr(6, 26);
    if (whichline == "00000000000000000000000000000000") {
        // nop
        mipsline += "nop\n";
    }
    else if (whichline == "010000000000000000000000000011000") {
        // eret
        mipsline += "eret\n";
    }
    else if (OpCode == "000000") {
        // special
        if (Funct == "100000") {
            // add
            if (overflowtest) {
                mipsline += "add ";
            }
            else {
                mipsline += "addu ";
            }
        }
        mipsline += "$";
        mipsline += ubinToDec(rd);
        mipsline += ", $";
        mipsline += ubinToDec(rs);
        mipsline += ", $";
        mipsline += ubinToDec(rt);
        mipsline += "\n";
    }
    else if (Funct == "100100") {
        // and
        mipsline += "and ";
        mipsline += "$";
        mipsline += ubinToDec(rd);
        mipsline += ", $";
        mipsline += ubinToDec(rs);
        mipsline += ", $";
        mipsline += ubinToDec(rt);
        mipsline += "\n";
    }
    else if (Funct == "011010") {

```

```
        // div
        mipsline += "div ";
        mipsline += "$";
        mipsline += ubinToDec(rs);
        mipsline += ", $";
        mipsline += ubinToDec(rt);
        mipsline += "\n";
    }
    else if (Funct == "011011") {
        // divu
        mipsline += "divu ";
        mipsline += "$";
        mipsline += ubinToDec(rs);
        mipsline += ", $";
        mipsline += ubinToDec(rt);
        mipsline += "\n";
    }
    else if (Funct == "001000") {
        //jr
        mipsline += "jr ";
        mipsline += "$";
        mipsline += ubinToDec(rs);
        mipsline += "\n";
    }
    else if (Funct == "010000") {
        // mfhi
        mipsline += "mfhi ";
        mipsline += "$";
        mipsline += ubinToDec(rd);
        mipsline += "\n";
    }
    else if (Funct == "010010") {
        // mflo
        mipsline += "mflo ";
        mipsline += "$";
        mipsline += ubinToDec(rd);
        mipsline += "\n";
    }
    else if (Funct == "010001") {
        // mthi
        mipsline += "mthi ";
        mipsline += "$";
        mipsline += ubinToDec(rs);
        mipsline += "\n";
    }
    else if (Funct == "010011") {
        // mtlo
        mipsline += "mtlo ";
        mipsline += "$";
        mipsline += ubinToDec(rs);
        mipsline += "\n";
    }
    else if (Funct == "011000") {
        // mult
```

```
        mipsline += "mult ";
        mipsline += "$";
        mipsline += ubinToDec(rs);
        mipsline += ", $";
        mipsline += ubinToDec(rt);
        mipsline += "\n";
    }
    else if (Funct == "011001") {
        // multu
        mipsline += "multu ";
        mipsline += "$";
        mipsline += ubinToDec(rs);
        mipsline += ", $";
        mipsline += ubinToDec(rt);
        mipsline += "\n";
    }
    else if (Funct == "100101") {
        // or
        mipsline += "or ";
        mipsline += "$";
        mipsline += ubinToDec(rd);
        mipsline += ", $";
        mipsline += ubinToDec(rs);
        mipsline += ", $";
        mipsline += ubinToDec(rt);
        mipsline += "\n";
    }
    else if (Funct == "101010") {
        // slt
        mipsline += "slt ";
        mipsline += "$";
        mipsline += ubinToDec(rd);
        mipsline += ", $";
        mipsline += ubinToDec(rs);
        mipsline += ", $";
        mipsline += ubinToDec(rt);
        mipsline += "\n";
    }
    else if (Funct == "101011") {
        // sltu
        mipsline += "sltu ";
        mipsline += "$";
        mipsline += ubinToDec(rd);
        mipsline += ", $";
        mipsline += ubinToDec(rs);
        mipsline += ", $";
        mipsline += ubinToDec(rt);
        mipsline += "\n";
    }
    else if (Funct == "100010") {
        // sub
        if (overflowtest) {
            mipsline += "sub ";
        }
    }
```



```

        else {
            mipsline += "subu ";
        }
        mipsline += "$";
        mipsline += ubinToDec(rd);
        mipsline += ", $";
        mipsline += ubinToDec(rs);
        mipsline += ", $";
        mipsline += ubinToDec(rt);
        mipsline += "\n";
    }
    else if (Funct == "001100") {
        // syscall
        mipsline += "syscall\n";
    }
    else {
        // unknown(using it to debug)
        mipsline += "unknown\n";
    }
}
else {
    if (OpCode == "001000") {
        // addi
        if (overflowtest) {
            mipsline += "addi ";
        }
        else {
            mipsline += "addiu ";
        }
        mipsline += "$";
        mipsline += ubinToDec(rt);
        mipsline += ", $";
        mipsline += ubinToDec(rs);
        mipsline += ", ";
        mipsline += binToDec(imm16);
        mipsline += "\n";
    }
    else if (OpCode == "001100") {
        // andi
        mipsline += "andi ";
        mipsline += "$";
        mipsline += ubinToDec(rt);
        mipsline += ", $";
        mipsline += ubinToDec(rs);
        mipsline += ", ";
        mipsline += ubinToDec(imm16);
        mipsline += "\n";
    }
    else if (OpCode == "000100") {
        // beq(see more in test-design.md to know how it works)
        int nowlabelcnt;
        int wherelabel = mipsCodes.size() + 1;
        wherelabel += binToDecInt(imm16);
        if (labels.size() <= wherelabel) {

```

```

        labels.resize(wherelabel + 1);
    }
    if (!labels[wherelabel]) {
        labels[wherelabel] = (++labelcnt);
        nowlabelcnt = labelcnt;
    }
    else {
        nowlabelcnt = labels[wherelabel];
    }
    mipsline += "beq ";
    mipsline += "$";
    mipsline += ubinToDec(rs);
    mipsline += ", $";
    mipsline += ubinToDec(rt);
    mipsline += ", ";
    mipsline += label;
    mipsline += to_string(nowlabelcnt);
    mipsline += "\n";
}
else if (OpCode == "000101") {
    // bne(see more in test-design.md to know how it works)
    int nowlabelcnt;
    int wherelabel = mipsCodes.size() + 1;
    wherelabel += binToDecInt(imm16);
    if (labels.size() <= wherelabel) {
        labels.resize(wherelabel + 1);
    }
    if (!labels[wherelabel]) {
        labels[wherelabel] = (++labelcnt);
        nowlabelcnt = labelcnt;
    }
    else {
        nowlabelcnt = labels[wherelabel];
    }
    mipsline += "bne ";
    mipsline += "$";
    mipsline += ubinToDec(rs);
    mipsline += ", $";
    mipsline += ubinToDec(rt);
    mipsline += ", ";
    mipsline += label;
    mipsline += to_string(nowlabelcnt);
    mipsline += "\n";
}
else if (OpCode == "100000") {
    // lb
    mipsline += "lb ";
    mipsline += "$";
    mipsline += ubinToDec(rt);
    mipsline += ", ";
    mipsline += binToDec(imm16);
    mipsline += "($";
    mipsline += ubinToDec(rs);
    mipsline += ")\n";
}

```

```

    }
    else if (OpCode == "100001") {
        // lh
        mipsline += "lh ";
        mipsline += "$";
        mipsline += ubinToDec(rt);
        mipsline += ", ";
        mipsline += binToDec(imm16);
        mipsline += "($";
        mipsline += ubinToDec(rs);
        mipsline += ")\n";
    }
    else if (OpCode == "001111") {
        // lui
        mipsline += "lui ";
        mipsline += "$";
        mipsline += ubinToDec(rt);
        mipsline += ", ";
        mipsline += ubinToDec(imm16);
        mipsline += "\n";
    }
    else if (OpCode == "100011") {
        // lw
        mipsline += "lw ";
        mipsline += "$";
        mipsline += ubinToDec(rt);
        mipsline += ", ";
        mipsline += binToDec(imm16);
        mipsline += "($";
        mipsline += ubinToDec(rs);
        mipsline += ")\n";
    }
    else if (OpCode == "000010") {
        //j
        int nowlabelcnt;
        int wherelabel = binToDecInt(imm26) - 3072; // (3072 == (0x3000 >>
2))

        if (labels.size() <= wherelabel) {
            labels.resize(wherelabel + 1);
        }
        if (!labels[wherelabel]) {
            labels[wherelabel] = (++labelcnt);
            nowlabelcnt = labelcnt;
        }
        else {
            nowlabelcnt = labels[wherelabel];
        }
        mipsline += "j ";
        mipsline += label;
        mipsline += to_string(nowlabelcnt);
        mipsline += "\n";
    }
    else if (OpCode == "000011") {
        //jal

```

2))

```

        int nowlabelcnt;
        int wherelabel = binToDecInt(imm26) - 3072; // (3072 == (0x3000 >>

        if (labels.size() <= wherelabel) {
            labels.resize(wherelabel + 1);
        }
        if (!labels[wherelabel]) {
            labels[wherelabel] = (++labelcnt);
            nowlabelcnt = labelcnt;
        }
        else {
            nowlabelcnt = labels[wherelabel];
        }
        mipsline += "jal ";
        mipsline += label;
        mipsline += to_string(nowlabelcnt);
        mipsline += "\n";
    }
    else if (OpCode == "001101") {
        // ori
        mipsline += "ori ";
        mipsline += "$";
        mipsline += ubinToDec(rt);
        mipsline += ", $";
        mipsline += ubinToDec(rs);
        mipsline += ", ";
        mipsline += ubinToDec(imm16);
        mipsline += "\n";
    }
    else if (OpCode == "101000") {
        // sb
        mipsline += "sb ";
        mipsline += "$";
        mipsline += ubinToDec(rt);
        mipsline += ", ";
        mipsline += binToDec(imm16);
        mipsline += "($";
        mipsline += ubinToDec(rs);
        mipsline += ")\n";
    }
    else if (OpCode == "101001") {
        // sh
        mipsline += "sh ";
        mipsline += "$";
        mipsline += ubinToDec(rt);
        mipsline += ", ";
        mipsline += binToDec(imm16);
        mipsline += "($";
        mipsline += ubinToDec(rs);
        mipsline += ")\n";
    }
    else if (OpCode == "101011") {
        // sw
        mipsline += "sw ";

```

```

        mipsline += "$";
        mipsline += ubinToDec(rt);
        mipsline += ", ";
        mipsline += binToDec(imm16);
        mipsline += "($";
        mipsline += ubinToDec(rs);
        mipsline += ")\n";
    }
    else if (OpCode == "010000") {
        if (rs == "00000") {
            // mfc0
            mipsline += "mfc0 ";
            mipsline += "$";
            mipsline += ubinToDec(rt);
            mipsline += ", $";
            mipsline += ubinToDec(rd);
            mipsline += "\n";
        }
        else if (rs == "00100") {
            // mtc0
            mipsline += "mtc0 ";
            mipsline += "$";
            mipsline += ubinToDec(rt);
            mipsline += ", $";
            mipsline += ubinToDec(rd);
            mipsline += "\n";
        }
        else {
            // unknown(using it to debug)
            mipsline += "unknown\n";
        }
    }
    else {
        // unknown(using it to debug)
        mipsline += "unknown\n";
    }
    mipsCodes.push_back(mipsline);
}

// print codes and labels(see more in test-design.md to know how it works)
for (int i = 0; i < mipsCodes.size(); i++) {
    if (i < labels.size() && labels[i]) {
        outputFile << label + to_string(labels[i]) + ":\n";
    }
    outputFile << mipsCodes[i];
}
for (int i = mipsCodes.size(); i < labels.size(); i++) {
    outputFile << label + to_string(labels[i]) + ":\n";
}

outputFile.close();

```

```
    return 0;
}
```

## 思考题答案：

Q:上面我们介绍了通过 FSM 理解单周期 CPU 的基本方法。请大家指出单周期 CPU所用到的模块中，哪些发挥状态存储功能，哪些发挥状态转移功能： 状态存储功能：GRF、PC、IM、DM 状态转移功能：NPC、ALU、EXT、Control

ROM是只读存储器，断电后指令不丢失，不会别其他操作更改，非常适合用来储存指令；RAM既可以读也可以写，因此满足DM对读写的要求；GRF是寄存器堆，需要较高的读写速度，因此适合用寄存器实现。。

Register是硬件层面速度最快的存储单元，直接与运算单元ALU相连，能实现“零延迟”数据交互。

Q：在上述提示的模块之外，你是否在实际实现时设计了其他的模块？如果是的话，请给出介绍和设计的思路。 否，但是添加了一些指令

nop是默认状态 没有任何一个使能信号为1 所以不会对grf和DM造成任何影响

评价pre中的mips测试代码： pre中的mips测试代码中，未涵盖 sub等算术指令以及 bne等分支跳转指令，所以核心指令集不完整 缺边界样例数据，add 未测试溢出情况，lui 未单独检测低 16 位清零效果，未使用 \$sp 等特殊寄存器，缺少 32 位有符号数极值，覆盖维度不全面。lw sw偏移量未测试负值